COURS POUR DÉBUTANT·E·S · AUCUNE CONNAISSANCE REQUISE

# Créer un site web de A à Z

Comprendre comment un vrai site est construit, mis en ligne et géré au quotidien — en ouvrant, fichier par fichier, celui de **The Evergreen Project**.

Site : dimogdor.github.io/Evergreen · Code source : github.com/Dimogdor/Evergreen

**COMMENT LIRE CE COURS**

Chaque chapitre explique une idée simplement, puis montre **où la voir dans le site** (encadré vert « À regarder ») et propose parfois un petit **essai** (encadré orange). Les encadrés bleus « À dire » sont des phrases-clés pour l'explication à l'oral. Pas besoin de tout retenir : l'objectif est de comprendre *qui fait quoi*.

# Sommaire


1. Comment fonctionne un site web
2. Les trois langages du web : HTML, CSS, JavaScript
3. La boîte à outils
4. Le dépôt GitHub : le dossier du projet
5. Astro : le « générateur » de site
6. Séparer le contenu du code (YAML)
7. Le design concrètement : de l'idée à l'écran
8. Mise en page et responsive (mobile, tablette, ordinateur)
9. Photos et vidéos
10. Un site en 8 langues
11. Les fonctionnalités, une par une
12. L'espace d'administration (sans code)
13. Le petit serveur : le Worker
14. Être trouvé sur Google (SEO)
15. Accessibilité : un site pour tout le monde
16. Sécurité
17. Rapidité (performance)
18. Tester son site
19. Mettre en ligne : hébergement, domaine, DNS
20. Le cycle de vie d'une modification
21. Par où commencer pour apprendre
22. Glossaire


**À DIRE EN INTRODUCTION**

« Un site web, c'est juste des fichiers texte qu'un navigateur sait lire. Tout le reste — les outils, les services — sert à écrire ces fichiers plus vite, à les rendre beaux, et à les mettre en ligne. »

# 1 · Comment fonctionne un site web

Quand on tape une adresse dans un navigateur (Chrome, Safari, Firefox…), il se passe trois choses :

Navigateur → **demande** → Serveur (un ordinateur toujours allumé) → **renvoie des fichiers** → Le navigateur les affiche

1. **L'adresse** (URL), par ex. `https://dimogdor.github.io/Evergreen/fr/contact/`, indique quel serveur contacter et quelle page demander.
2. **Le serveur** renvoie des fichiers : une page **HTML**, des feuilles de style **CSS**, des scripts **JavaScript**, des images.
3. **Le navigateur** assemble tout et affiche la page.

## Site statique ou dynamique ?

| | Site statique (notre cas) | Site dynamique (ex. WordPress) |
|---|---|---|
| Principe | Les pages sont fabriquées **à l'avance**, une fois, puis simplement envoyées. | Chaque page est fabriquée **à chaque visite** par un programme et une base de données. |
| Vitesse | Très rapide | Plus lent |
| Sécurité | Très peu de failles possibles | Mises à jour de sécurité fréquentes |
| Coût | Hébergement gratuit | Serveur payant en général |

Le site Evergreen est **statique** : à chaque modification, un outil fabrique toutes les pages (171 !) en quelques secondes, puis on les dépose sur l'hébergeur. On verra plus loin comment on garde quand même un espace d'administration et des formulaires.

> **À REGARDER**
>
> Ouvrir le site, puis clic droit → **« Afficher le code source de la page »** : c'est le HTML que le serveur a envoyé. Ensuite clic droit → **« Inspecter »** : ce sont les **outils de développement** du navigateur, le meilleur ami de tout créateur de site.

# 2 · Les trois langages du web

Tous les sites du monde reposent sur trois langages, chacun avec un rôle précis. Une image simple : une maison.

| Langage | Rôle | Dans la maison |
|---|---|---|
| **HTML** | La **structure** et le contenu : titres, paragraphes, images, liens, formulaires. | Les murs et les pièces |
| **CSS** | L'**apparence** : couleurs, polices, tailles, espacements, disposition. | La peinture et la déco |
| **JavaScript** | Le **comportement** : ce qui réagit quand on clique, ce qui change sans recharger. | L'électricité, les interrupteurs |

## HTML : des « balises »

```
<h1>Contact</h1>                      ← un grand titre
<p>Écrivez-nous, nous répondons vite.</p>  ← un paragraphe
<a href="mailto:evergreenecovillage@gmail.com">Nous écrire</a>  ← un lien
<img src="tente.webp" alt="Une tente dans la forêt">          ← une image + sa description
```

## CSS : des « règles »

```
h1 { font-family: 'EB Garamond'; color: #22402a; }   ← tous les grands titres en vert foncé
.btn { border-radius: 999px; padding: 0.85em 1.4em; } ← les boutons arrondis
```

## JavaScript : des « instructions »

```
bouton.addEventListener('click', () => {               ← quand on clique sur le bouton…
  navigator.clipboard.writeText('GR7701103910000039100269800');  ← …copier l'IBAN
});
```

**À REGARDER DANS LE DÉPÔT**

- CSS : `src/styles/global.css` — tout le style du site dans un seul fichier.
- JavaScript : en bas de `src/pages/[lang]/[page].astro`, le bouton « Copier » de l'IBAN (lignes `<script>`).
- HTML : partout dans les fichiers `.astro` (voir chapitre 5).

## Et TypeScript ?

C'est du JavaScript avec des « étiquettes » qui précisent le type des données (texte, nombre…). Ça évite des erreurs. Les fichiers `.ts` du projet (ex. `src/calendar.ts`) en sont.

**À DIRE**

« Commence par HTML et CSS : en deux semaines tu peux faire une vraie page. JavaScript vient après, quand tu veux que la page réagisse. »

# 3 · La boîte à outils

| Outil | À quoi ça sert | Gratuit ? |
|---|---|---|
| **Un éditeur de code** : VS Code | Écrire et organiser les fichiers, avec couleurs et suggestions. | Oui |
| **Un navigateur** + ses outils de dév (F12) | Voir le résultat, inspecter, tester sur « faux » téléphone. | Oui |
| **Le terminal** | Taper des commandes (lancer le site, installer des outils). | Oui |
| **Node.js** et **npm** | Node fait tourner JavaScript sur l'ordinateur ; npm installe des « paquets » (outils écrits par d'autres). | Oui |
| **Git** | Garder l'historique de chaque modification (comme « Annuler », mais pour toujours). | Oui |
| **GitHub** | Stocker le projet en ligne, collaborer, déclencher la mise en ligne automatique. | Oui |
| **Figma** (optionnel) | Dessiner les maquettes avant de coder. | Oui (offre de base) |

## Les trois commandes du projet

```
npm install     ← télécharge les outils listés dans package.json (une seule fois)
npm run dev     ← lance le site sur l'ordinateur : http://localhost:4321 (se met à jour à chaque sauvegarde)
npm run build   ← fabrique la version finale dans le dossier dist/
```

**À REGARDER**

`package.json` : la « carte d'identité » du projet. On y voit les commandes ( `scripts` ) et les paquets utilisés ( `dependencies` ) : **astro** (le générateur), **@astrojs/sitemap** (plan du site pour Google), **marked** (transforme du texte simple en HTML), **yaml** (lit les fichiers de contenu), et deux polices (**Inter** et **EB Garamond**).
`.node-version` : la version de Node à utiliser (22).

## Git en 5 mots

- **Dépôt** (repository) : le dossier du projet avec tout son historique.
- **Commit** : une « photo » des fichiers à un instant, avec un message (« Inhabitants: new photo of our dogs »).
- **Branche** : une copie de travail parallèle pour essayer sans casser la version officielle ( `main` ).
- **Pull request** : la demande de fusionner une branche dans `main` , avec relecture.
- **Push / pull** : envoyer / récupérer les commits vers / depuis GitHub.

**ESSAI**

Sur github.com/Dimogdor/Evergreen, cliquer sur **« Commits »** : on voit chaque changement fait sur le site, qui l'a fait et quand, et on peut afficher exactement les lignes modifiées.

# 4 · Le dépôt : le plan du projet

Voici l'organisation des dossiers. Ranger proprement, c'est la moitié du travail.

```
Evergreen/
├── package.json            ← carte d'identité, commandes, outils
├── astro.config.mjs        ← réglages du générateur (adresse du site, sécurité)
├── README.md               ← mode d'emploi du projet
├── docs/GUIDE-ADMIN.md     ← mode d'emploi pour les administrateurs
├── public/                 ← fichiers copiés tels quels
│   ├── admin/              ← l'espace d'administration (config.yml, index.html)
│   ├── uploads/videos/     ← vidéos
│   └── _headers            ← règles de sécurité/cache pour l'hébergeur
├── src/                    ← le cœur du site
│   ├── pages/              ← une page = un fichier (chapitre 5)
│   ├── layouts/Base.astro  ← le cadre commun : en-tête, menu, pied de page
│   ├── components/         ← briques réutilisables (image, liste d'événements…)
│   ├── styles/global.css   ← tout le design
│   ├── content/            ← TOUT le texte du site, en fichiers YAML (chapitre 6)
│   ├── assets/uploads/     ← photos d'origine (optimisées automatiquement)
│   ├── lib.ts              ← petites fonctions utiles (lire le contenu, les liens…)
│   ├── i18n.ts             ← les langues
│   └── calendar.ts         ← calculs de dates, fuseaux horaires, fichiers agenda
├── worker/                 ← le petit serveur (connexion admin, témoignages)
├── tests/                  ← vérifications automatiques
└── .github/workflows/      ← robots de GitHub : mise en ligne, compression vidéo
```

**À DIRE**

« Le code (pages, design) et le contenu (textes, photos) sont dans des dossiers séparés. Les responsables du lieu ne touchent qu'au contenu, via l'admin ; le code, lui, ne bouge presque jamais. »

# 5 · Astro : le « générateur » de site

Écrire 171 pages HTML à la main serait impossible (8 langues × une vingtaine de pages). On utilise donc un **générateur de site statique** : on écrit **un modèle** par type de page, et l'outil le remplit avec chaque contenu, dans chaque langue. Ici, c'est **Astro** (astro.build).

## Un fichier .astro = deux parties

```
---
// Partie 1, entre les tirets : du JavaScript qui prépare les données (s'exécute pendant la
fabrication)
const page = await one('pages', 'contact', lang);
---
<!-- Partie 2 : du HTML, dans lequel on insère les données entre accolades -->
<h1>{page.title}</h1>
<p class="lead">{page.intro}</p>
```

## Les pages = les adresses

Dans `src/pages/`, le nom du fichier devient l'adresse. Les crochets `[ ]` veulent dire « variable » :

| Fichier | Adresses produites |
|---|---|
| `src/pages/[lang]/index.astro` | /el/, /en/, /fr/… (les 8 pages d'accueil) |
| `src/pages/[lang]/[page].astro` | /fr/contact/, /fr/place/, /fr/support/… (toutes les pages de texte) |
| `src/pages/[lang]/activities/[slug].astro` | /fr/activities/retreats/… (une page par activité) |
| `src/pages/[lang]/calendar.astro` | /fr/calendar/ |
| `src/pages/[lang]/gallery.astro`, `stories.astro` | la galerie, les témoignages |
| `src/pages/calendar.ics.ts` | /calendar.ics : un fichier agenda, pas une page ! |

## Le layout : le cadre commun

`src/layouts/Base.astro` contient ce qui est identique sur toutes les pages : le `<head>` invisible (titre de l'onglet, description pour Google, icône), l'en-tête avec le logo et le menu, le choix de langue, le pied de page avec les contacts. Chaque page « s'emboîte » dedans avec `<Base> … </Base>`.

## Les composants : des briques LEGO

Le plus simple pour comprendre : ouvrir `src/components/Story.astro` (12 lignes) — une photo, une citation (`<blockquote>`), le nom et le pays.

| Composant | Ce qu'il fait |
|---|---|
| `src/components/Img.astro` | Affiche n'importe quelle photo, optimisée automatiquement (chapitre 9). |
| `Blocks.astro` | Les sections « titre + texte + photo » qui alternent gauche/droite. |
| `EventList.astro` | La liste d'événements avec dates, prix, bouton « Ajouter à mon agenda ». |
| `Story.astro` | Une carte de témoignage. |
| `Video.astro` | Une vidéo YouTube/Vimeo qui ne se charge qu'au clic. |

# 6 · Séparer le contenu du code

Tous les textes, dates, prix et chemins de photos sont rangés dans `src/content/`, au format **YAML** : un format texte très lisible, fait de « clé : valeur » et de retraits.

> **À REGARDER : UN ÉVÉNEMENT**
>
> `src/content/events/essence-of-yoga-2026.yml`

```
el:                                  ← la version grecque (langue principale)
  start: '2026-11-28'                ← date de début
  end: '2026-12-05'                  ← date de fin
  category: retreat
  image: /src/assets/uploads/yin-shala.webp
  link: https://bookretreats.com/r/8-day-essence-of-yoga-…
  title: Essence of Yoga — χειμερινό εντατικό …
  price: 380 € early bird (κράτηση έως 28/10/2026) · 450 € κανονική τιμή
fr:                                  ← en français : seulement ce qui se traduit
  title: Essence of Yoga — immersion d'hiver avec Panos Katsandris
  price: 380 € early bird (réservation avant le 28/10/2026) · 450 € plein tarif
```

| Dossier | Contenu |
|---|---|
| `settings/site.yml` | Nom, e-mail, téléphone, réseaux, IBAN, couleurs du site |
| `settings/ui.yml` | Les petits textes de l'interface (menu, boutons) dans les 8 langues |
| `pages/` | Le texte de chaque page (accueil, le lieu, mode de vie…) |
| `programs/` | Les activités (volontariat, retraites…) |
| `events/` | Le calendrier : un fichier par événement |
| `inhabitants/` | Les habitants : Katerina, Kostas, les chiens, les chats, les poules… |
| `albums/`, `testimonials/`, `extra/` | Galerie, témoignages, pages créées dans l'admin |

Comment le code lit ce contenu ? `src/content.config.ts` déclare les dossiers, et la fonction `list()` de `src/lib.ts` les lit et choisit la bonne langue.

> **À DIRE**
>
> « Pas de base de données : le contenu, ce sont de simples fichiers texte rangés dans des dossiers. On peut les lire, les sauvegarder, revenir en arrière. L'admin ne fait qu'éditer ces fichiers pour nous. »

# 7 · Le design concrètement : de l'idée à l'écran

## Étape 1 — Collecter des inspirations (« moodboard »)

Avant de coder, on regarde des sites qu'on aime et on note *pourquoi*. Pour Evergreen : kaleidoscope.media (grands titres élégants, beaucoup d'espace, numéros de sections « 01 — ») et evergreenyogastudios.com (identité de la famille Evergreen). On récupère aussi la matière : logo, photos récentes, ton des textes.

## Étape 2 — Choisir une palette

4 à 6 couleurs suffisent, tirées du sujet : ici le papier crème, la mousse, la forêt profonde, la terre cuite. On vérifie le **contraste** (texte lisible sur le fond) avec un outil comme *webaim.org/resources/contrastchecker*.

```
:root {                       ← src/styles/global.css, tout en haut
  --paper: #f4f0e6;           fond crème
  --ink: #17231a;             texte presque noir, teinté de vert
  --moss: #3d673f;            vert mousse
  --moss-deep: #22402a;       vert forêt (bandeau CES, pied de page)
  --clay: #b0622a;            terre cuite (touches d'accent)
}
```

Ces « variables » (`--paper`…) sont réutilisées partout ; changer une ligne change tout le site. C'est même modifiable depuis l'admin (Réglages → Couleurs).

## Étape 3 — Choisir deux polices

Une **police à empattements** pour les titres (EB Garamond : chaleureuse, « livre ») et une **police simple** pour le texte (Inter : très lisible sur écran). Elles viennent de Google Fonts mais sont **installées dans le projet** via les paquets `@fontsource-variable/…` : aucun appel à Google, plus rapide et plus respectueux de la vie privée.

## Étape 4 — Une échelle d'espacements et de tailles

Plutôt que des tailles fixes, on utilise `clamp(min, idéal, max)` : le titre grandit avec l'écran sans jamais devenir trop petit ou trop grand. Exemple : `--gutter: clamp(16px, 4vw, 56px)` = la marge sur les côtés.

## Étape 5 — Des « composants visuels » cohérents

Boutons ronds en majuscules espacées (`.btn`), petites étiquettes (`.label`), cartes photo + titre (`.card`), filets horizontaux numérotés (`.rule`). Une fois définis, on les réutilise : le site paraît « dessiné », pas bricolé.

## Étape 6 — Maquette (optionnel) puis code

Dans Figma on dessine la page d'accueil en version ordinateur et téléphone. Puis on la traduit en HTML (la structure) et en CSS (le style) — et on ajuste directement dans le navigateur avec les outils de dév.

> **ESSAI**
>
> Ouvrir le site, F12 → onglet « Éléments », cliquer sur `<html>` et changer `--moss-deep` en `#5a2a6e` : tout le site change de couleur en direct (rien n'est cassé, il suffit de recharger).

# 8 · Mise en page et responsive

Un même site doit s'afficher bien sur un téléphone (390 px de large) comme sur un grand écran (1440 px et plus). C'est le **responsive design**.

## Les deux outils de mise en page du CSS moderne

| Outil | Pour quoi | Exemple dans le site |
|---|---|---|
| **Flexbox** | Aligner des éléments sur une ligne (ou une colonne). | L'en-tête : logo à gauche, menu à droite (`.site-header`). |
| **Grid** | Une grille en lignes et colonnes. | Les cartes d'activités, les habitants (`.grid.cols-3`, `.cols-4`). |

```
.cols-3 { grid-template-columns: repeat(auto-fill, minmax(min(100%, 340px), 1fr)); }
← « autant de colonnes de 340 px minimum que la largeur permet » : 3 sur ordinateur, 1 sur
téléphone, sans rien d'autre

@media (max-width: 760px) {                          ← « si l'écran fait moins de 760 px… »
  .event { grid-template-columns: 1fr; }              ← …les événements passent sur une seule
colonne
  .event .thumb { order: -1; }                        ← …et la photo passe au-dessus du texte
}
```

Les **media queries** (`@media`) sont des « si… alors… » selon la taille d'écran. Mais la grille « auto-adaptative » du premier exemple évite souvent d'en écrire : c'est aussi le cas du pied de page (`minmax(180px, 1fr)`).

## Le menu sur téléphone

Sur petit écran le menu se replie derrière un bouton « Menu ». Le code est dans `src/layouts/Base.astro` (bouton `.menu-btn`) et le style dans `global.css`.

## Les détails qui font « pro »

- `text-wrap: balance` : les titres sur deux lignes ont des lignes de longueur égale.
- `@media (prefers-reduced-motion)` : on coupe les animations pour les personnes que ça gêne.
- Une largeur maximale (`--max: 1440px`) pour que le texte ne s'étale pas sur un écran géant.

> **ESSAI**
>
> F12 → icône téléphone/tablette (« mode appareil ») → choisir « iPhone » : voir le site se réorganiser. Élargir la fenêtre petit à petit et observer quand les colonnes changent.

# 9 · Photos et vidéos

Les images sont souvent 80 % du poids d'une page. Une photo de téléphone pèse 4 à 8 Mo ; sur le site, la même photo pèse 50 à 200 Ko. Comment ?

## Trois optimisations automatiques

1. **À l'envoi dans l'admin** : la photo est réduite à 2000 px maximum et convertie en **WebP**, un format plus léger que JPEG (réglage `transformations` dans `src/admin/config.yml`).
2. **À la fabrication du site** : Astro crée **5 tailles** de chaque photo (480, 800, 1200, 1600, 1920 px). Fichier : `src/components/Img.astro`.
3. **Dans le navigateur** : grâce aux attributs `srcset` et `sizes`, un téléphone télécharge la petite version, un grand écran la grande. Et `loading="lazy"` : une photo n'est téléchargée que quand on s'en approche en faisant défiler.

## Le texte alternatif

Chaque image a un `alt` : une description lue aux personnes aveugles et utilisée par Google. Ex. sur l'accueil : « Relaxation finale dans la shala de l'Evergreen Project » (réglable dans l'admin).

## Les vidéos

- **YouTube / Vimeo** (recommandé) : on colle le lien ; le composant `Video.astro` n'affiche qu'une miniature et ne charge le lecteur qu'au clic (version « youtube-nocookie », sans cookies) → page rapide et respectueuse.
- **Fichier MP4** : un robot GitHub (`.github/workflows/compress-videos.yml`) le recompresse avec l'outil **ffmpeg** (format H.264, 1280 px max).

## La visionneuse plein écran

Dans `src/pages/[lang]/gallery.astro`, un clic sur une photo l'ouvre en grand dans un élément HTML natif `<dialog>` — pas besoin de bibliothèque externe : le navigateur sait déjà gérer la fenêtre, la touche Échap et le focus.

# 10 · Un site en 8 langues

Grec (langue par défaut), anglais, français, espagnol, italien, russe, allemand, portugais. Trois mécanismes :

1. **Les adresses** commencent par la langue : `/el/`, `/fr/`… Le dossier `[lang]` dans `src/pages/` génère chaque page dans chaque langue. La liste des langues est dans `src/i18n.ts`.
2. **Le contenu** : chaque fichier YAML a une partie par langue (chapitre 6).
3. **Les petits textes** (« Menu », « Lire la suite », « Ajouter à mon agenda »…) : `src/content/settings/ui.yml`, 96 phrases × 8 langues.

## Le « repli » (fallback)

Si une phrase n'est pas traduite dans une langue, le site affiche l'anglais, sinon le grec. On ne voit jamais de trou. Le code : fonction `localize` dans `src/lib.ts`.

## Dire à Google qu'il existe des traductions

Dans le `<head>` de chaque page (`Base.astro`), des balises `<link rel="alternate" hreflang="fr">` indiquent les versions dans les autres langues : un visiteur français tombe sur la page française.

> **À REGARDER**
>
> Sur le site, le sélecteur de langue en haut à droite : il garde la même page et change seulement le préfixe ( `/fr/contact/` → `/de/contact/` ).

# 11 · Les fonctionnalités, une par une

Pour chaque fonctionnalité : ce qu'elle fait, l'outil ou le service utilisé, et le fichier à ouvrir.

| Fonctionnalité | Comment c'est fait | Où regarder |
|---|---|---|
| **Calendrier mensuel** | Un petit script JavaScript dessine le mois et colore les jours occupés ; les événements passés disparaissent tout seuls (comparaison avec la date du jour). | `src/pages/[lang]/calendar.astro`, `src/components/EventList.astro` |
| **« Ajouter à mon agenda »** (Google) | Un simple lien vers Google Agenda contenant le titre, les dates et le fuseau « Europe/Athens » : Google convertit à l'heure du visiteur. | `src/calendar.ts` (fonction `googleUrl`) |
| **Apple / Outlook (.ics)** | Un fichier `.ics` : format texte standard (norme RFC 5545) que tous les agendas comprennent. Les heures sont converties en temps universel avec l'outil intégré du navigateur/JavaScript `Intl`. | `src/calendar.ts` (fonction `ics`), `src/pages/[lang]/events/[id].ics.ts` |
| **S'abonner au calendrier** | Un fichier `/calendar.ics` avec tous les événements ; l'agenda du visiteur le relit régulièrement. | `src/pages/calendar.ics.ts` |
| **Carte** | Une carte **OpenStreetMap** (libre, gratuite) intégrée dans un `<iframe>`, plus un lien « Ouvrir dans Maps » vers Google Maps. | `src/pages/[lang]/[page].astro` (partie contact) |
| **Formulaire de contact** | Service **Web3Forms** : le formulaire envoie les champs à leur serveur, qui transfère par e-mail. Une case invisible piège les robots (« honeypot »). | même fichier, `<form action="https://api.web3forms.com/submit">` |
| **Don / IBAN** | Affichage de l'IBAN par groupes de 4 + bouton qui le copie (API **Clipboard** du navigateur). | même fichier, bloc `donate` + `<script>` en bas |
| **Témoignages** | Formulaire → petit serveur (Worker, chapitre 13) → enregistré « non publié » → un admin valide. | `src/pages/[lang]/stories.astro`, `worker/index.js` |
| **Galerie** | Albums en « mosaïque » CSS, visionneuse `<dialog>`. | `src/pages/[lang]/gallery.astro` |
| **Bannière d'annonce** | Si le champ « Ανακοίνωση » de l'admin est rempli, une bande orange apparaît en haut de toutes les pages. | `Base.astro` (`site.announcement`) |
| **Lien vers les studios de yoga** | Un lien externe (pas de doublon : cours et tarifs restent sur leur site). | `src/content/settings/site.yml` |
| **Mise en forme des textes** | Les textes longs sont écrits en **Markdown** (`**gras**`, `- liste`, `[lien](adresse)`), transformés en HTML par le paquet **marked**. | `src/lib.ts` (fonction `md`) |

# 12 · L'espace d'administration (sans code)

Katerina et Kostas ne vont pas éditer des fichiers YAML. On leur donne donc une **interface** à l'adresse `/admin/` : des formulaires, des boutons, du glisser-déposer de photos. C'est un **CMS** (Content Management System = système de gestion de contenu).

## Le CMS choisi : Sveltia CMS

- Gratuit et open source, tourne entièrement dans le navigateur.
- Il lit et écrit directement les fichiers YAML du dépôt : **pas de base de données**.
- Gère nativement les 8 langues côte à côte, compresse les photos à l'envoi.

## Comment on le configure

Deux fichiers seulement :

- `public/admin/index.html` : charge le programme Sveltia (version figée + empreinte de sécurité, voir chapitre 16).
- `src/admin/config.yml` : décrit **chaque formulaire**. Pour chaque champ : son nom, son étiquette, son type (« widget »).

```
- { name: start, label: Έναρξη · Start date, widget: datetime, type: date }   ← un calendrier pour choisir la date
- { name: price, label: Τιμή · Price, widget: string, i18n: true }          ← un champ texte, traduisible
- { name: image, label: Φωτογραφία · Photo, widget: image }                ← un bouton d'envoi de photo
- { name: published, label: Published, widget: boolean }                   ← une case à cocher
```

## Ce qui se passe quand on clique sur « Save »

Admin : Save → Le fichier YAML est modifié dans le dépôt → Un robot refabrique le site → En ligne ~2 min plus tard

Chaque sauvegarde est un « commit » : on peut toujours revenir à une version précédente.

## L'aide à l'écriture

Un bouton **« ✓ Correct »** apparaît à côté du champ en cours : il propose une version corrigée (orthographe, grammaire, ponctuation), à accepter ou non. Et à chaque enregistrement, les textes grecs nouveaux ou modifiés sont **traduits automatiquement** dans les 7 autres langues. Les deux passent par le Worker (chapitre 13), qui appelle un service de correction et de traduction automatiques. Code : `public/admin/index.html` (le bouton) et `worker/ai.js`.

> **À REGARDER**
>
> `docs/GUIDE-ADMIN.md` : le mode d'emploi rédigé pour les administrateurs (grec + anglais). Bon exemple de documentation pour non-techniciens.

# 13 · Le petit serveur : le Worker

Un site statique ne peut pas garder de secret ni recevoir de données (il n'y a pas de programme qui tourne sur le serveur). Pour deux besoins, on ajoute donc un **tout petit serveur**, gratuit, chez **Cloudflare** : un **Worker** (une fonction JavaScript qui s'exécute quand on l'appelle).

| Besoin | Ce que fait le Worker |
|---|---|
| **Connexion des admins** | Affiche un formulaire e-mail + mot de passe, vérifie, puis délivre un « badge » de session (valable 30 jours), signé avec une clé secrète (signature HMAC) pour qu'il soit impossible à falsifier. |
| **Sauvegardes de l'admin** | Relaie les demandes de l'admin vers le stockage en y ajoutant la vraie clé d'accès, que seul le Worker connaît. |
| **Aide à l'écriture** | Corrige un texte à la demande ; traduit chaque enregistrement dans les 7 langues avant de le ranger. |
| **Témoignages** | Reçoit le formulaire, filtre le spam, vérifie que la photo est vraiment une image, enregistre le récit « non publié » et prévient l'équipe. |

Les « secrets » (mots de passe des admins, clé d'accès) sont rangés chez Cloudflare, chiffrés — **jamais** dans le code du dépôt.

> **À REGARDER**
> `worker/index.js` (bien commenté, découpé en sections : connexion, relais, témoignages) · `worker/wrangler.toml` (réglages) · `worker/README.md` (installation pas à pas). L'outil **wrangler** ( `npx wrangler deploy` ) envoie le Worker chez Cloudflare.

## API : le mot à connaître

Une **API**, c'est une « porte » qu'un programme ouvre à d'autres programmes : on envoie une demande à une adresse, on reçoit une réponse. Le site en utilise plusieurs :

| API | Utilisée pour |
|---|---|
| API GitHub | Lire / écrire les fichiers de contenu (via le Worker) |
| Service de traduction / correction | Traduire et corriger les textes de l'admin (via le Worker) |
| Web3Forms | Envoyer le formulaire de contact par e-mail |
| Cloudflare Turnstile (optionnel) | Anti-robots, alternative respectueuse aux « CAPTCHA » |
| APIs du navigateur | Presse-papiers (copier l'IBAN), `Intl` (dates et fuseaux), `<dialog>` |
| Google Agenda (lien) | Ajouter un événement à son agenda |

# 14 · Être trouvé sur Google (SEO)

Le SEO (référencement naturel), c'est aider les moteurs de recherche à comprendre et à bien présenter le site.

| Élément | Rôle | Où |
|---|---|---|
| `<title>` et `<meta name="description">` | Le titre bleu et les deux lignes de texte dans les résultats Google. | `Base.astro` |
| Open Graph (`og:image`…) | L'aperçu (photo + titre) quand on partage le lien sur Facebook, WhatsApp… | `Base.astro` |
| Plan du site (`sitemap-index.xml`) | La liste de toutes les pages pour Google ; généré par **@astrojs/sitemap**. | `astro.config.mjs` |
| `robots.txt` | Dit aux robots ce qu'ils peuvent visiter (pas l'admin). | `src/pages/robots.txt.ts` |
| Données structurées (JSON-LD) | Une fiche lisible par les machines : « ceci est une ONG, voici son adresse, son téléphone » (vocabulaire **schema.org**). | `Base.astro` (`ld`) |
| `hreflang` | Les traductions de chaque page. | `Base.astro` |
| Adresses lisibles | `/fr/contact/` plutôt que `/page?id=37`. | noms des fichiers dans `src/pages/` |

**ESSAI**

Coller l'adresse du site dans *opengraph.xyz* pour voir l'aperçu de partage, ou chercher « Rich Results Test » de Google pour voir la fiche JSON-LD.

# 15 · Accessibilité : un site pour tout le monde

Environ une personne sur cinq a un handicap (vue, motricité, audition, attention…). Un site accessible est aussi plus agréable pour tous, et mieux classé par Google.

- **Textes alternatifs** des images (`alt`).
- **Contrastes** suffisants : texte gris clair sur fond crème = illisible pour beaucoup. Le crédit en pied de page a été foncé pour cette raison.
- **Titres dans l'ordre** (`h1` puis `h2`…) : les lecteurs d'écran s'en servent comme d'un sommaire.
- **Lien « Aller au contenu »** caché, visible au clavier (`.skip` dans `Base.astro`) : évite de traverser tout le menu.
- **Navigation au clavier** : contour visible sur l'élément sélectionné (`:focus-visible` dans `global.css`).
- **Noms explicites** pour les boutons et liens sans texte (`aria-label`) : par ex. les photos-liens de l'accueil.
- Respect du réglage « réduire les animations » du téléphone.

> **ESSAI**
>
> Sur le site, appuyer plusieurs fois sur la touche **Tab** : on doit toujours voir où l'on est. Puis F12 → onglet **Lighthouse** → « Accessibilité » → Analyser.

# 16 · Sécurité

| Risque | Protection dans ce site | Où |
|---|---|---|
| Qu'un pirate injecte un script dans une page | **CSP** (Content-Security-Policy) : une liste blanche de ce que la page a le droit de charger ; tout le reste est bloqué par le navigateur. | `astro.config.mjs` (`security.csp`) |
| Qu'un service tiers (le CMS) soit modifié à notre insu | **Version figée + SRI** : une empreinte du fichier ; s'il change d'un seul octet, le navigateur refuse de l'exécuter. | `public/admin/index.html` (`integrity="sha384-…"`) |
| Fuite de mots de passe / clés | Secrets uniquement chez Cloudflare, jamais dans le dépôt ; fichiers de secrets locaux ignorés par Git. | `.gitignore`, `worker/README.md` |
| Essais de mots de passe en rafale, spam | Limite de 3 tentatives / minute / visiteur, champ piège, délai minimum, Turnstile en option. | `worker/index.js`, `wrangler.toml` |
| Faux fichiers envoyés | Vérification des premiers octets (« signature ») : un JPEG commence toujours par `FF D8 FF`. | `worker/index.js` (`looksLike`) |
| Espionnage de la connexion | HTTPS partout (le cadenas). | hébergeur |
| Site affiché dans un cadre piégé | En-tête `X-Frame-Options`. | `public/_headers` |
| Pistage des visiteurs | Aucun cookie, aucun outil de statistiques, polices et vidéos sans traceurs. | page Confidentialité |

**À DIRE**

« La meilleure sécurité, c'est d'avoir peu de choses à protéger : un site statique n'a ni base de données ni mot de passe à voler. Les seuls secrets sont rangés à un seul endroit. »

# 17 · Rapidité (performance)

Un site lent fait fuir (et Google le pénalise). Les recettes appliquées ici :

- **Pages déjà prêtes** (statique) : rien à calculer à la visite.
- **Images** : WebP, plusieurs tailles, chargement différé (chapitre 9). La première grande photo est, elle, chargée en priorité ( `fetchpriority="high"` ).
- **Très peu de JavaScript** : seulement de petits scripts là où c'est utile. Astro n'envoie aucun JavaScript par défaut.
- **Polices hébergées sur le site** au lieu de Google Fonts.
- **Vidéos chargées au clic**.
- **Mise en cache** : les fichiers dont le nom contient une empreinte (dossier `_astro/` ) sont gardés un an par le navigateur ( `public/_headers` ).

Résultat mesuré avec **Lighthouse** (outil intégré à Chrome, onglet « Lighthouse ») sur mobile : performance 98–99/100, page d'accueil ≈ 256 Ko.

## Les indicateurs à connaître

| Indicateur | Signification | Objectif |
|---|---|---|
| LCP | Temps d'affichage du plus grand élément (souvent la photo principale) | < 2,5 s |
| CLS | La page « saute »-t-elle pendant le chargement ? | < 0,1 |
| INP / TBT | La page répond-elle vite aux clics ? | le plus bas possible |

# 18 · Tester son site

## Les tests automatiques

Des petits programmes vérifient que le code fait ce qu'il doit. Ici avec l'outil intégré à Node (`node:test`), lancés par `npm test` :

- `tests/calendar.test.ts` : « 10 h à Athènes en décembre = 8 h en temps universel, mais 7 h en juillet (heure d'été) », « le fichier .ics est bien formé »…
- `tests/worker.test.ts` : « un mauvais mot de passe est refusé », « un faux fichier image est rejeté », « la clé secrète n'arrive jamais dans le navigateur »…

## Les vérifications manuelles (à chaque grosse modification)

| Quoi | Comment |
|---|---|
| Affichage | Sur un vrai téléphone + le mode appareil des outils de dév ; Chrome, Safari, Firefox. |
| Liens cassés | Cliquer partout, ou un outil de vérification de liens. |
| Accessibilité | Lighthouse, extension **axe DevTools**, navigation au clavier. |
| Erreurs | F12 → onglet « Console » : aucune ligne rouge. |
| Contenu | Relire dans chaque langue (idéalement par des natifs). |

# 19 · Mettre en ligne : hébergement, domaine, DNS

## L'hébergement

Le dossier `dist/` fabriqué par `npm run build` est déposé chez un **hébergeur**. Pour un site statique, des services gratuits suffisent : **GitHub Pages** (utilisé aujourd'hui), **Cloudflare Pages** (prévu pour la suite), Netlify…

## Le robot de mise en ligne

`.github/workflows/deploy.yml` est une recette que GitHub exécute automatiquement à chaque modification de `main` : installer Node → `npm ci` → `npm run build` → publier `dist/`. C'est l'**intégration / déploiement continu** (CI/CD) : personne n'a besoin de « mettre en ligne » à la main.

## Le nom de domaine

Un nom comme `evergreenproject.gr` s'achète (≈ 10–15 €/an) chez un **registrar** (Papaki, DNHOST, OVH…). On le loue à l'année, on ne le possède pas : attention au renouvellement !

## Le DNS : l'annuaire d'Internet

Le DNS traduit un nom (`evergreenproject.gr`) en adresse de serveur. On y ajoute des « enregistrements » :

- **A** : « ce nom pointe vers ce serveur (adresse IP) ».
- **CNAME** : « ce nom est un alias de cet autre nom » (ex. `www` → l'hébergeur).
- **MX** : « les e-mails de ce domaine vont ici ».

> **À REGARDER**
>
> Dans `README.md`, sections « Nom de domaine propre » et « Quitter GitHub » : les étapes concrètes pour brancher un domaine et changer d'hébergeur.

# 20 · Le cycle de vie d'une modification

## Côté développeur (changer le design ou une fonctionnalité)

1. Créer une **branche** (copie de travail).
2. Modifier les fichiers dans VS Code ; vérifier en direct avec `npm run dev`.
3. Lancer les tests (`npm test`) et la fabrication (`npm run build`).
4. **Commit** avec un message clair, **push** vers GitHub.
5. Ouvrir une **pull request**, relire, puis **fusionner** dans `main`.
6. Le robot met en ligne : ~2 minutes plus tard, c'est visible pour tout le monde.

## Côté administrateur (changer un texte, une date, une photo)

1. Ouvrir `/admin/`, se connecter.
2. Modifier, cliquer sur **Save**.
3. ~2 minutes plus tard, c'est en ligne. Rien à installer.

**EXEMPLE RÉEL**

Dans l'historique du dépôt (« Commits »), chercher « Inhabitants: new photo of our dogs » : on y voit une nouvelle photo ajoutée dans `src/assets/uploads/` et une seule ligne changée dans `src/content/inhabitants/dogs.yml` — c'est exactement ce que fait l'admin quand on change une photo.

# 21 · Par où commencer pour apprendre

## Un parcours conseillé (environ 3 mois à quelques heures par semaine)

| Étape | Objectif | Ressources gratuites |
|---|---|---|
| 1. HTML | Faire une page avec titres, textes, images, liens | MDN Web Docs (« Apprendre le développement web »), freeCodeCamp |
| 2. CSS | Couleurs, polices, puis Flexbox et Grid | Jeux **Flexbox Froggy** et **Grid Garden**, web.dev/learn/css |
| 3. Responsive | Une page qui s'adapte au téléphone | web.dev/learn/design |
| 4. Git & GitHub | Sauvegarder et publier son travail | GitHub Skills, « Learn Git Branching » |
| 5. JavaScript | Ajouter un peu d'interaction (bouton, menu) | javascript.info (en français), freeCodeCamp |
| 6. Un générateur | Refaire un petit site avec Astro | docs.astro.build (tutoriel « Créer un blog ») |
| 7. Publier | Mettre son site en ligne gratuitement | GitHub Pages, Cloudflare Pages |

## Petits exercices sur ce site (sans risque, en local)

1. Changer la couleur `--clay` dans `src/styles/global.css` et observer.
2. Ajouter un faux événement : copier un fichier de `src/content/events/`, changer les dates et le titre.
3. Modifier le texte d'un bouton dans `src/content/settings/ui.yml`.
4. Ajouter un habitant (un animal !) dans `src/content/inhabitants/`.
5. Lire `src/components/Story.astro` puis essayer de créer son propre composant.

> **CONSEIL FINAL**
>
> « On n'apprend pas en lisant, on apprend en cassant puis en réparant. Avec Git, rien n'est jamais vraiment cassé : on peut toujours revenir en arrière. »

# 22 · Glossaire

| | |
|---|---|
| **API** | Porte d'accès qu'un service ouvre à d'autres programmes. |
| **Build** | La fabrication du site final (dossier `dist/`). |
| **Cache** | Copie gardée par le navigateur pour ne pas retélécharger. |
| **CI/CD** | Robots qui testent et mettent en ligne automatiquement. |
| **CMS** | Interface pour gérer le contenu sans coder (ici Sveltia, à `/admin/`). |
| **Commit** | Une modification enregistrée dans l'historique, avec un message. |
| **Composant** | Brique de page réutilisable (`Img`, `Story`…). |
| **CSP** | Liste blanche de sécurité de ce qu'une page peut charger. |
| **DNS** | L'annuaire qui traduit un nom de domaine en adresse de serveur. |
| **Dépôt (repo)** | Le dossier du projet avec tout son historique (sur GitHub). |
| **Front-end / back-end** | Ce qui s'exécute chez le visiteur / sur le serveur. |
| **Hébergeur** | Service qui garde les fichiers du site en ligne 24h/24. |
| **i18n** | « Internationalisation » (i + 18 lettres + n) : gestion des langues. |
| **Layout** | Le cadre commun à toutes les pages. |
| **Markdown** | Façon simple d'écrire du texte formaté (`**gras**`). |
| **npm / paquet** | Le magasin d'outils JavaScript / un outil qu'on y télécharge. |
| **Pull request** | Demande de fusionner une branche, avec relecture. |
| **Responsive** | Qui s'adapte à toutes les tailles d'écran. |
| **SEO** | Référencement : être bien trouvé et présenté par Google. |
| **SRI** | Empreinte qui garantit qu'un fichier externe n'a pas été modifié. |
| **Statique** | Pages fabriquées à l'avance plutôt qu'à chaque visite. |
| **URL** | L'adresse d'une page. |
| **WebP** | Format d'image moderne, plus léger que JPEG. |
| **Worker** | Petite fonction serveur hébergée chez Cloudflare. |
| **YAML** | Format texte lisible (« clé : valeur ») qui stocke le contenu. |